BASI DI DATI

# Algebra relazionale

Daniele Riboni · Università degli Studi di Cagliari
Dipartimento di Matematica e Informatica

*Slide tratte in parte dal materiale didattico del libro: Atzeni, Ceri, Fraternali, Paraboschi, Torlone. Basi di dati. McGraw-Hill*

DEFINIZIONI

# Algebra e algebra relazionale

## Algebra (in generale)

Una branca della matematica che studia come rappresentare informazione numerica tramite simboli, e le regole per manipolarli.

## Algebra relazionale

Un'algebra per modellare dati di strutture relazionali ed esprimere interrogazioni.

MOTIVAZIONE

# Perché l'algebra relazionale?

## Le origini

- Definita nei primi anni '70 da Edgar F. Codd, alla IBM
- Fondamento teorico dei linguaggi di interrogazione per basi di dati relazionali, tra cui SQL
- Base per implementare e ottimizzare i DBMS reali

## Perché la studiamo

- La studiamo di pari passo con SQL
- Ci aiuta a capire come scrivere le query
- Gli operatori "di base" sono gli stessi; SQL ne aggiunge altri per query più complesse

LINGUAGGI DI INTERROGAZIONE

# Dichiarativi e procedurali

**DICHIARATIVI**

Specificano le proprietà del risultato: **che cosa** si vuole ottenere.

Esempio: SQL (parzialmente)

**PROCEDURALI**

Specificano le modalità di generazione del risultato: **come** ottenerlo.

Esempio: algebra relazionale

ALGEBRA RELAZIONALE

# Che cos'è l'algebra relazionale?

Un insieme di operatori...

**1**

Operano **su relazioni**

**2**

**Producono relazioni** come risultato

**3**

Possono essere **composti** fra loro

PANORAMICA

# Gli operatori dell'algebra relazionale

### Operatori insiemistici

Unione, intersezione, differenza

### Ridenominazione

Cambia nome ad attributi e relazioni

### Selezione e proiezione

Filtrano tuple e attributi

### Join

Join naturale, prodotto cartesiano, theta-join, equi-join

CLASSI DI OPERATORI

# Operatori insiemistici

Le relazioni sono **insiemi** (di ennuple)

I risultati degli operatori devono essere **relazioni**

Unione, intersezione e differenza si possono applicare **solo a relazioni definite sugli stessi attributi**

# Unione

**Laureati**

| Matricola | Nome | Età |
|---|---|---|
| 7274 | Rossi | 42 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

∪

**Specialisti**

| Matricola | Nome | Età |
|---|---|---|
| 9297 | Bianchi | 33 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

=

**Laureati ∪ Specialisti**

| Matricola | Nome | Età |
|---|---|---|
| 7274 | Rossi | 42 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |
| 9297 | Bianchi | 33 |

**Una relazione è un insieme**: le tuple duplicate (9824/Verdi/45, 7432/Neri/54) compaiono una sola volta nel risultato

# Differenza

**Laureati**

| Matricola | Nome | Età |
|---|---|---|
| 7274 | Rossi | 42 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

−

**Specialisti**

| Matricola | Nome | Età |
|---|---|---|
| 9297 | Blu | 33 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

=

**Laureati – Specialisti**

| Matricola | Nome | Età |
|---|---|---|
| 7274 | Rossi | 42 |

Il risultato contiene le tuple di Laureati che **non** compaiono in Specialisti

# Intersezione

**Laureati**

| Matricola | Nome | Età |
|---|---|---|
| 7274 | Rossi | 42 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

∩

**Specialisti**

| Matricola | Nome | Età |
|---|---|---|
| 9297 | Blu | 33 |
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

=

**Laureati ∩ Specialisti**

| Matricola | Nome | Età |
|---|---|---|
| 7432 | Gialli | 54 |
| 9824 | Verdi | 45 |

Il risultato contiene le tuple che compaiono **sia** in Laureati **sia** in Specialisti

OPERATORI INSIEMISTICI

# Un'unione sensata, ma impossibile

**Paternità(Padre,Figlio)**

| Padre | Figlio |
|---|---|
| Adamo | Abele |
| Adamo | Caino |
| Abramo | Isacco |

∪

**Maternità(Madre,Figlio)**

| Madre | Figlio |
|---|---|
| Eva | Abele |
| Eva | Set |
| Sara | Isacco |

=

**??**

Concettualmente ha senso unire "chi è genitore di chi" — ma **l'unione non è definita**: le due relazioni hanno schemi diversi (Padre ≠ Madre)

RIDENOMINAZIONE

# Ridenominazione

Mi permette di cambiare nome agli **attributi** e alle **relazioni**

Modifica **lo schema**, lasciando inalterate le **istanze**

RIDENOMINAZIONE — ESEMPIO

# Rinominare per poter confrontare

`REN Genitore←Padre(Paternità)`

| Genitore | Figlio |
|---|---|
| Adamo | Abele |
| Adamo | Caino |
| Abramo | Isacco |

`REN Genitore←Madre(Maternità)`

| Genitore | Figlio |
|---|---|
| Eva | Abele |
| Eva | Set |
| Sara | Isacco |

Ora entrambe le relazioni rinominate hanno lo stesso schema: (Genitore,Figlio)

RIDENOMINAZIONE — RISULTATO

# L'unione ora è possibile

```
( REN Genitore←Padre (Paternità) ) ∪ ( REN Genitore←Madre (Maternità) )
```

**(Genitore,Figlio)**

| Genitore | Figlio |
|---|---|
| Adamo | Abele |
| Adamo | Caino |
| Abramo | Isacco |
| Eva | Abele |
| Eva | Set |
| Sara | Isacco |

RIDENOMINAZIONE — ATTRIBUTI MULTIPLI

# Rinominare più attributi insieme

**Impiegati(Cognome,Ufficio,Stipendio)**

| Cognome | Ufficio | Stipendio |
|---|---|---|
| Rossi | Roma | 55 |
| Neri | Milano | 64 |

Un'unica operazione **REN** può rinominare più attributi contemporaneamente

**REN** Sede,Retrib ← Ufficio,Stipendio(Impiegati)

**Operai(Cognome,Fabbrica,Salario)**

| Cognome | Fabbrica | Salario |
|---|---|---|
| Bruni | Monza | 45 |
| Verdi | Latina | 55 |

**REN** Sede,Retrib ← Fabbrica,Salario (Operai)

RIDENOMINAZIONE — RISULTATO

# Ora Impiegati e Operai si possono unire

```
( REN Sede,Retrib←Ufficio,Stipendio (Impiegati) ) ∪
( REN Sede,Retrib←Fabbrica,Salario (Operai) )
```

**(Cognome,Sede,Retrib)**

| Cognome | Sede | Retrib |
|---|---|---|
| Rossi | Roma | 55 |
| Neri | Milano | 64 |
| Bruni | Monza | 45 |
| Verdi | Latina | 55 |

RIDENOMINAZIONE — SINTASSI

# Notazione

NOTAZIONE A PAROLE

**REN** Sede,Retribuzione ← Fabbrica,Salario (Operai)

NOTAZIONE MATEMATICA EQUIVALENTE

$\rho$ Sede,Retribuzione ← Fabbrica,Salario(Operai)

RIDENOMINAZIONE — SINTASSI

# Sintassi alternativa

Quando voglio ridenominare le colonne e anche il nome della relazione

Data la relazione E (o una generica espressione) avente n attributi, rinomino E in R e i suoi attributi in $A_1$, $A_2$, ..., $A_n$

$$\rho\ R(A_1, A_2, \ldots, A_n)\ (E)$$

**ESEMPIO: Ridenominazione di ESAME(Studente, Corso, Voto)**

$$\rho\ \text{APPELLO}(\text{Matricola, CodC, Esito})(\text{ESAME})$$

OPERATORE ALGEBRICO

# Selezione

Un "**filtro**" sulle tuple, in base a condizioni sui valori

**Il risultato...**

Ha lo **stesso schema** dell'operando

Contiene un **sottoinsieme delle tuple** dell'operando: quelle che soddisfano la condizione

SELEZIONE — ESEMPIO

# Impiegati che...

**Impiegati(Matricola,Cognome,Filiale,Stipendio)**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Rossi | Roma | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Milano | Milano | 44 |
| 5698 | Neri | Napoli | 64 |

1. ...guadagnano più di 50
2. ...guadagnano più di 50 e lavorano a Milano
3. ...hanno lo stesso nome della filiale presso cui lavorano

SELEZIONE

# Selezione: sintassi e semantica

SINTASSI

```
SEL Condizione (Espressione)
```

Condizione: un'espressione booleana (come quelle dei vincoli di ennupla)

SEMANTICA

Il risultato contiene le ennuple dell'operando che **soddisfano la condizione**

SELEZIONE — ESEMPIO

# Impiegati che guadagnano più di 50

```
SEL Stipendio>50 (Impiegati)
```

**Impiegati**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Rossi | Roma | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Milano | Milano | 44 |
| 5698 | Neri | Napoli | 64 |

→

**Risultato**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Rossi | Roma | 55 |
| 5998 | Neri | Milano | 64 |
| 5698 | Neri | Napoli | 64 |

SELEZIONE — ESEMPIO

## Impiegati che guadagnano più di 50 e lavorano a Milano

```
SEL Stipendio>50 AND Filiale='Milano'(Impiegati)
```

**Impiegati**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Rossi | Roma | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Milano | Milano | 44 |
| 5698 | Neri | Napoli | 64 |

→

**Risultato**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 5998 | Neri | Milano | 64 |

SELEZIONE — SINTASSI ALTERNATIVA

# Sintassi alternativa

Le due notazioni sono equivalenti

**NOTAZIONE A PAROLE**

```
SEL Stipendio>50 AND Filiale='Milano' (Impiegati)
```

**NOTAZIONE MATEMATICA EQUIVALENTE (SIGMA)**

$$\sigma \text{ Stipendio}>50 \wedge \text{Filiale}=\text{'Milano'} \ (\text{Impiegati})$$

SELEZIONE

# Impiegati con lo stesso cognome della filiale

```
SEL Cognome=Filiale (Impiegati)
```

**Impiegati**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Rossi | Roma | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Milano | Milano | 44 |
| 5698 | Neri | Napoli | 64 |

**Risultato**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 9553 | Milano | Milano | 44 |

# Selezione con valori nulli

**SEL** Età>40 (Impiegati)

**Impiegati**

| Matricola | Cognome | Filiale | Età |
|---|---|---|---|
| 7309 | Rossi | Roma | 32 |
| 5998 | Neri | Milano | 45 |
| 9553 | Bruni | Milano | NULL |

**Risultato**

| Matricola | Cognome | Filiale | Età |
|---|---|---|---|
| 5998 | Neri | Milano | 45 |

La tupla con Età **NULL** sparisce: la condizione atomica è vera solo per valori non nulli.

VALORI NULLI — LA SELEZIONE

# Un risultato non desiderabile

**SEL** Età>30 (Persone) ∪ SEL Età<=30 (Persone) **≠ Persone**

**Perché?** Le due selezioni vengono valutate **separatamente**: una tupla con Età NULL non soddisfa né la prima né la seconda condizione, e non compare in nessuna delle due metà dell'unione.

**SEL** (Età>30 ∨ Età<=30) (Persone) **≠ Persone**

**Perché?** Anche **dentro** una singola selezione, le condizioni atomiche vengono valutate separatamente: ciascun confronto con NULL resta sconosciuto, non diventa vero.

VALORI NULLI — LA SOLUZIONE

# IS NULL e IS NOT NULL

La condizione atomica è vera solo per valori non nulli. Per riferirsi esplicitamente ai valori nulli esistono forme apposite di condizione:

`IS NULL`

`IS NOT NULL`

**Ora funziona**

$$\text{SEL}_{\text{Età}>30}(\text{Persone}) \cup \text{SEL}_{\text{Età}<=30}(\text{Persone}) \cup \text{SEL}_{\text{Età IS NULL}}(\text{Persone})$$

$$= \text{SEL}_{(\text{Età}>30 \vee \text{Età}<=30 \vee \text{Età IS NULL})}(\text{Persone}) = \textbf{Persone}$$

VALORI NULLI — UN ESEMPIO COMPLETO

## Includere esplicitamente il caso nullo

```
SEL (Età>40) OR (Età IS NULL) (Impiegati)
```

**Impiegati**

| Matricola | Cognome | Filiale | Età |
|---|---|---|---|
| 7309 | Rossi | Roma | 32 |
| 5998 | Neri | Milano | 45 |
| 9553 | Bruni | Milano | NULL |

**Risultato**

| Matricola | Cognome | Filiale | Età |
|---|---|---|---|
| 5998 | Neri | Milano | 45 |
| 9553 | Bruni | Milano | NULL |

Ora la tupla con Età NULL entra nel risultato perché è coperta esplicitamente dalla condizione **IS NULL**

SELEZIONE E PROIEZIONE

# Due filtri ortogonali

Selezione e proiezione sono operatori ortogonali: agiscono su due dimensioni diverse della stessa relazione.

SELEZIONE

## Filtraggio orizzontale

Sceglie un sottoinsieme delle ENNUPLE (righe) della relazione.

PROIEZIONE

## Filtraggio verticale

Sceglie un sottoinsieme degli ATTRIBUTI (colonne) della relazione.

PROIEZIONE

# Cosa produce la proiezione

- Produce un risultato che ha PARTE degli attributi (colonne) dell'operando
- Contiene ennuple cui contribuiscono TUTTE le ennuple dell'operando

PROIEZIONE

# Un nuovo esempio: Impiegati

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Neri | Napoli | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Rossi | Roma | 44 |
| 5698 | Rossi | Roma | 64 |

**Per tutti gli impiegati, vogliamo conoscere:**

1. Matricola e cognome
2. Cognome e filiale

PROIEZIONE

# Sintassi e semantica

### Sintassi

```
PROJ ListaAttributi (Operando)
```

### Semantica

Il risultato contiene le ennuple ottenute da tutte le ennuple dell'operando, ristrette agli attributi presenti nella lista

**PROIEZIONE**

# Matricola e cognome di tutti gli impiegati

```
PROJ Matricola,Cognome(Impiegati)
```

**Impiegati**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Neri | Napoli | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Rossi | Roma | 44 |
| 5698 | Rossi | Roma | 64 |

**Risultato**

| Matricola | Cognome |
|---|---|
| 7309 | Neri |
| 5998 | Neri |
| 9553 | Rossi |
| 5698 | Rossi |

## PROIEZIONE

# Cognome e filiale di tutti gli impiegati

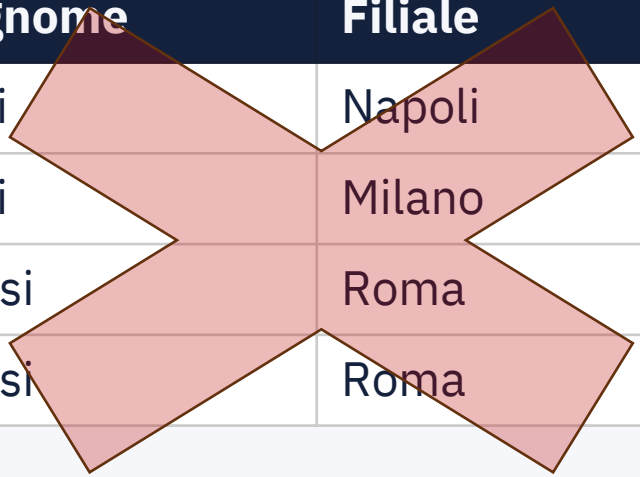
```
PROJ Cognome,Filiale(Impiegati)
```

**Impiegati**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Neri | Napoli | 55 |
| 5998 | Neri | Milano | 64 |
| 9553 | Rossi | Roma | 44 |
| 5698 | Rossi | Roma | 64 |

**Risultato “grezzo” – è corretto?**

| Cognome | Filiale |
|---|---|
| Neri | Napoli |
| Neri | Milano |
| Rossi | Roma |
| Rossi | Roma |



**PROIEZIONE**

# Cognome e filiale: attenzione ai duplicati

```
PROJ Cognome,Filiale (Impiegati)
```

**Proiezione grezza (4 ennuple)**

| Cognome | Filiale |
|---|---|
| Neri | Napoli |
| Neri | Milano |
| Rossi | Roma |
| Rossi | Roma |

**Relazione risultato (insieme, 3 ennuple)**

| Cognome | Filiale |
|---|---|
| Neri | Napoli |
| Neri | Milano |
| Rossi | Roma |

**Le relazioni sono insiemi**: le due ennuple duplicate (in rosso) collassano in una sola.

SINTASSI ALTERNATIVA

# La lettera greca pi

```
PROJ Cognome,Filiale (Impiegati)
```

è equivalente a

```
π Cognome,Filiale (Impiegati)
```

PROIEZIONE

# Cardinalità delle proiezioni

Una proiezione contiene AL PIÙ tante ennuple quante l'operando

**⚠ Può contenerne di meno!!**

PROIEZIONE QUIZ

## Cardinalità delle proiezioni

`PROJ K (R)`

Se K è una superchiave di R, quante tuple contiene questa proiezione?

**A** Può contenere meno tuple di R

**B** Può contenere più tuple di R

**C** Esattamente tante tuple quanto R

PROIEZIONE QUIZ

## Cardinalità delle proiezioni

`PROJ K (R)`

Se K è una superchiave di R, quante tuple contiene questa proiezione?

A Può contenere meno tuple di R

B Può contenere più tuple di R

C **Esattamente tante tuple quanto R**

SELEZIONE E PROIEZIONE

# Combinando selezione e proiezione possiamo possiamo estrarre informazioni interessanti da una da una relazione

SELEZIONE + PROIEZIONE

# Matricola e cognome di chi guadagna più di 50

```
PROJ Matricola,Cognome ( SEL Stipendio>50 (Impiegati))
```

Stessa relazione Impiegati vista in precedenza.

**Dopo SEL Stipendio>50**

| Matricola | Cognome | Filiale | Stipendio |
|---|---|---|---|
| 7309 | Rossi | Roma | 55 |
| 5998 | Neri | Milano | 64 |
| 5698 | Neri | Napoli | 64 |

→

**Risultato finale**

| Matricola | Cognome |
|---|---|
| 7309 | Rossi |
| 5998 | Neri |
| 5698 | Neri |

## SELEZIONE + PROIEZIONE: ATTENTI ALL'ORDINE DELLE OPERAZIONI !

# Matricola e cognome di chi guadagna più di 50

```
SEL Stipendio>50 ( PROJ Matricola,Cognome Impiegati)
```

Stessa relazione Impiegati vista in precedenza.

**Dopo PROJ Matricola,Cognome**

| Matricola | Cognome |
|---|---|
| 7309 | Neri |
| 5998 | Neri |
| 9553 | Rossi |
| 5698 | Rossi |

→ `SEL Stipendio>50`

**Risultato finale**



**ESERCIZIO 1 DI 6**

# Le città dei trentenni

**Schema**

**Persona** (CF, Età, CittàNatale, CittàAttuale)

**Domanda**

*In quali città abitano dei trentenni?*

**Soluzione**

**PROJ** CittàAttuale ( **SEL** Età=30 (Persona))

**Attenzione all'ordine delle operazioni! In questo caso non posso fare:**

~~**SEL** Età=30 ( **PROJ** CittàAttuale (Persona))~~

**ESERCIZIO 2 DI 6**

# Città natale senza residenti

**Schema**

**Persona** (CF, Età, CittàNatale, CittàAttuale)

**Domanda** *In quali città è nato qualcuno ma non abita nessuno?*

**Soluzione**

```
REN Città←CittàNatale PROJ CittàNatale (Persona) −
 REN Città←CittàAttuale PROJ CittàAttuale (Persona)
```

**ESERCIZIO 3 DI 6**

# Città natale con residenti (non necessariamente nati lì)

**Schema**

**Persona** (CF, Età, CittàNatale, CittàAttuale)

**Domanda** *Quali sono le città in cui è nato qualcuno e abita qualcuno?*

**Soluzione**

**REN** Città←CittàNatale **PROJ** CittàNatale (Persona) ∩
**REN** Città←CittàAttuale **PROJ** CittàAttuale (Persona)

ESERCIZIO 4 DI 6

# Città con nativi residenti

**Schema**

**Persona** (CF, Età, CittàNatale, CittàAttuale)

**Domanda**

*In quali città risiede qualche nativo? (nato lì e tuttora residente lì)*

**Soluzione — due notazioni equivalenti**

**PROJ** CittàNatale ( **SEL** CittàNatale=CittàAttuale (Persona))

$\pi$ CittàNatale ( $\sigma$ CittàNatale=CittàAttuale (Persona))

**ESERCIZIO 5 DI 6**

# Tutte le città

**Schema**

**Persona** (CF, Età, CittàNatale, CittàAttuale)

**Domanda**

*Quali sono tutte le città presenti nella base di dati?*

**Soluzione**

```
REN Città←CittàNatale PROJ CittàNatale (Persona) U
 REN Città←CittàAttuale PROJ CittàAttuale (Persona)
```

**ESERCIZIO 6 DI 6**

# Città di soli anziani

### Schema

**Persona** (CF, Età, CittàNatale, CittàAttuale)

### Domanda

*In quali città abitano solo anziani (età ≥ 70)?*

### Soluzione

**PROJ** CittàAttuale (Persona) **−** **PROJ** CittàAttuale ( **SEL** Età<70 (Persona))

SELEZIONE E PROIEZIONE

# I limiti di selezione e proiezione

Combinando selezione e proiezione possiamo estrarre informazioni da una relazione. Non possiamo però correlare informazioni presenti in relazioni diverse.



### Relazioni diverse

Non correlano dati che vivono in tabelle separate



### Ennuple diverse

Non confrontano righe diverse di una stessa relazione

UN NUOVO OPERATORE

# Il join

L'operatore più interessante dell'algebra relazionale

Permette di correlare dati in relazioni diverse

JOIN

# L'analogia del concorso pubblico

I compiti sono anonimi e ad ognuno è associata una busta chiusa con il nome del candidato.

**Compito anonimo**

**stesso numero**

**Busta col nome**

Ciascun compito e la relativa busta vengono contrassegnati con uno stesso numero.

JOIN

# Voti e candidati: l'accostamento intuitivo

### Voti

| Numero | Voto |
|---|---|
| 1 | 25 |
| 2 | 13 |
| 3 | 27 |
| 4 | 28 |

+

### Candidati

| Numero | Candidato |
|---|---|
| 1 | Mario Rossi |
| 2 | Nicola Russo |
| 3 | Mario Bianchi |
| 4 | Remo Neri |

→

### Cosa vogliamo ottenere alla fine

| Candidato | Voto |
|---|---|
| Mario Rossi | 25 |
| Nicola Russo | 13 |
| Mario Bianchi | 27 |
| Remo Neri | 28 |

JOIN NATURALE

# Cos'è il join naturale

- È un operatore BINARIO, generalizzabile a più relazioni
- Produce un risultato sull'UNIONE degli attributi degli operandi
- Ogni ennupla del risultato è costruita a partire da una tupla di ciascun operando

## JOIN NATURALE - SEMANTICA

Voti

| Numero | Voto |
|---|---|
| 1 | 25 |
| 2 | 13 |
| 3 | 27 |
| 4 | 28 |

Candidati

| Numero | Candidato |
|---|---|
| 1 | Mario Rossi |
| 2 | Nicola Russo |
| 3 | Mario Bianchi |
| 4 | Remo Neri |

Il join naturale tra due relazioni combina le tuple che hanno **valori uguali in tutte le colonne con lo stesso nome**, eliminando le colonne duplicate usate per il confronto

```
NAT_JOIN (Voti, Candidati)
```

**Risultato**

| Numero | Candidato | Voto |
|---|---|---|
| 1 | Mario Rossi | 25 |
| 2 | Nicola Russo | 13 |
| 3 | Mario Bianchi | 27 |
| 4 | Remo Neri | 28 |

*Numero* è l'attributo in comune: compare solo una volta nel risultato

## JOIN NATURALE

# Quando ogni tupla contribuisce al risultato: abbiamo un join completo

```
Impiegati NAT_JOIN Reparti
```

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| A | Mori |
| B | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| Rossi | A | Mori |
| Neri | B | Bruni |
| Bianchi | B | Bruni |

Ogni reparto citato in **Impiegati** compare anche in **Reparti**: **nessuna tupla viene esclusa** dal risultato — è un join completo.

## JOIN NATURALE

# Un join non completo

```
Impiegati NAT_JOIN Reparti
```

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| B | Mori |
| C | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| Neri | B | Mori |
| Bianchi | B | Mori |

Le righe evidenziate **non contribuiscono** al risultato: il reparto A di Rossi non ha corrispondenza in Reparti, e il reparto C non ha impiegati.

## JOIN NATURALE

# Un join vuoto

```
Impiegati NAT_JOIN Reparti
```

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| D | Mori |
| C | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| — | — | — |

Nessun codice di reparto in **Impiegati** (A, B) compare in **Reparti** (D, C): **il risultato ha zero tuple**.

## JOIN NATURALE

# Un join completo, con n×m ennuple

```
Impiegati NAT_JOIN Reparti
```

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | B |
| Neri | B |

**Reparti**

| Reparto | Capo |
|---|---|
| B | Mori |
| B | Bruni |

**Risultato (2×2 = 4 righe)**

| Impiegato | Reparto | Capo |
|---|---|---|
| Rossi | B | Mori |
| Rossi | B | Bruni |
| Neri | B | Mori |
| Neri | B | Bruni |

Il codice B compare due volte su entrambi i lati (non è chiave): **il risultato può avere fino a |R1|×|R2| tuple**, non solo |R1|.

SINTASSI ALTERNATIVA

## Il join naturale in notazione greca

```
Impiegati NAT_JOIN Reparti
```

è equivalente a

Impiegati ⋈ Reparti

Il simbolo ⋈ ("bowtie", papillon) è la notazione classica del join naturale nei testi di basi di dati

JOIN NATURALE

# Cardinalità del join

Relazione R1(A,B), relazione R2(B,C). Il join naturale di R1 e R2 contiene:

In generale, un numero di ennuple: $0 \leq |R1 \text{ JOIN } R2| \leq |R1| \cdot |R2|$

Se B è chiave in R2: $0 \leq |R1 \text{ JOIN } R2| \leq |R1|$

Se B è chiave in R2 **e** c'è un vincolo di integrità referenziale da R1 a R2: $|R1 \text{ JOIN } R2| = |R1|$

JOIN NATURALE

## Quando non tutte le tuple hanno corrispondenze

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| B | Mori |
| C | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| Neri | B | Mori |
| Bianchi | B | Mori |

Rossi e il reparto C sono evidenziati perché **vengono tagliate fuori** dal risultato: alcune tuple non contribuiscono mai al join

Se volessimo recuperare queste tuple, invece di perderle? Il **join esterno** risponde a questa domanda

JOIN ESTERNO

# Tre version del Join esterno (outer join)

Il join esterno estende, con **valori nulli**, le ennuple che verrebbero tagliate fuori da un join (interno). Esiste in tre versioni:

### Sinistro

Mantiene tutte le ennuple del **primo operando**, estendendole con valori nulli se necessario.

### Destro

Mantiene tutte le ennuple del **secondo operando**, estendendole con valori nulli se necessario.

### Completo

Mantiene tutte le ennuple di **entrambi gli operandi**, estendendole con valori nulli se necessario.

## JOIN ESTERNO

# Join esterno sinistro

`Impiegati` **`JOIN_LEFT`** `Reparti`

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| B | Mori |
| C | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| Neri | B | Mori |
| Bianchi | B | Mori |
| Rossi | A | NULL |

Rossi viene **mantenuto** anche se il reparto A non ha corrispondenza: Capo è esteso a NULL invece di scartare la tupla.

## JOIN ESTERNO

# Join esterno destro

`Impiegati` **`JOIN_RIGHT`** `Reparti`

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| B | Mori |
| C | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| Neri | B | Mori |
| Bianchi | B | Mori |
| NULL | C | Bruni |

La tupla di Reparti con codice C viene **mantenuta** anche se nessun impiegato è in quel reparto: Impiegato è esteso a NULL.

JOIN ESTERNO

## Join esterno completo

`Impiegati JOIN_FULL Reparti`

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Reparto | Capo |
|---|---|
| B | Mori |
| C | Bruni |

**Risultato**

| Impiegato | Reparto | Capo |
|---|---|---|
| Neri | B | Mori |
| Bianchi | B | Mori |
| Rossi | A | NULL |
| NULL | C | Bruni |

Il join completo è l'**unione** del risultato sinistro e del risultato destro: mantiene le tuple di entrambi gli operandi.

PRODOTTO CARTESIANO

# Prodotto cartesiano

Come nella teoria degli insiemi: il prodotto cartesiano di due relazioni combina ogni ennupla del primo operando con ogni ennupla del secondo

Contiene sempre un numero di tuple pari al prodotto delle cardinalità degli operandi: tutte le combinazioni sono ammesse.

$$|R1 \times R2| = |R1| \cdot |R2|$$

PRODOTTO CARTESIANO

## Un esempio: Impiegati × Reparti

### Impiegati

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

### Reparti

| Reparto | Capo |
|---|---|
| A | Mori |
| B | Bruni |

### Risultato (3×2 = 6 righe)

| Impiegato | Reparto | Reparto | Capo |
|---|---|---|---|
| Rossi | A | A | Mori |
| Rossi | A | B | Bruni |
| Neri | B | A | Mori |
| Neri | B | B | Bruni |
| Bianchi | B | A | Mori |
| Bianchi | B | B | Bruni |

Tutte le combinazioni sono presenti, anche quelle senza senso (Rossi/A abbinato a B/Bruni)

L'attributo **Reparto** compare **due volte con lo stesso nome**: il prodotto cartesiano non lo unifica come fa il join naturale.

SINTASSI ALTERNATIVA

# Il prodotto cartesiano in notazione greca

```
Impiegati JOIN Reparti
```

è equivalente a

Impiegati × Reparti

Il simbolo × (moltiplicazione) è la notazione classica del prodotto cartesiano

# Theta-join

Il prodotto cartesiano, in pratica, ha senso (quasi) solo se seguito da una selezione:

```
SEL Condizione (R1 JOIN R2)
```

**l'operazione si chiama theta-join e si indica con**

```
R1 JOIN Condizione R2
```

THETA-JOIN

## Perché "theta-join"?

La condizione è spesso una congiunzione (AND) di atomi di confronto **A1 θ A2**, dove θ è uno degli operatori di confronto: =, >, <, ≥, ≤, ≠.

È l'operatore di join che **useremo più spesso** nel resto del corso

EQUI-JOIN

# Equi-join

Se l'operatore di confronto nel theta-join è sempre l'**uguaglianza** (=), si parla di **equi-join**.

Nota bene: di solito ci serve usare l'**equi-join**, non il theta-join più generale

EQUI-JOIN

## Un esempio di equi-join

```
Impiegati JOIN Reparto=Codice Reparti
```

**Impiegati**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Reparti**

| Codice | Capo |
|---|---|
| A | Mori |
| B | Bruni |

**Risultato**

| Impiegato | Reparto | Codice | Capo |
|---|---|---|---|
| Rossi | A | A | Mori |
| Neri | B | B | Bruni |
| Bianchi | B | B | Bruni |

Stesso effetto del join naturale, ma **Reparto** e **Codice** restano **due colonne distinte**: l'equi-join non le unifica, ha nomi diversi.

SINTASSI ALTERNATIVA

## L'equi-join in notazione greca

```
Impiegati JOIN Reparto=Codice Reparti
```

è equivalente a

```
Impiegati ⋈ Reparto=Codice (Reparti)
```

Invece di = posso avere ≠, >, ≤, ..., e comporre più condizioni con AND (raramente OR)

SINTESI

## Il theta-join tramite il prodotto cartesiano

Impiegati(Impiegato,Reparto), Reparti(Codice,Capo).

```
Impiegati JOIN Reparto=Codice Reparti

=

SEL Reparto=Codice (Impiegati JOIN Reparti)
```

Il theta-join è, letteralmente, un prodotto cartesiano seguito da una selezione sulla condizione

ESEMPI

## Un nuovo schema per gli esempi che seguono

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

Matricola e Impiegato/Capo sono codici numerici; Stipendio è espresso in migliaia di euro. Queste due relazioni saranno usate negli esempi di interrogazione che seguono.

ALGEBRA RELAZIONALE — QUERY GUIDATE

# Query 1 — Chi guadagna più di 40

Impiegati(Matricola, Nome, Età, Stipendio)

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

Supervisione(Impiegato, Capo)

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

*"Trovare matricola, nome, età e stipendio degli impiegati che guadagnano più di 40."*

## Come ragionare

- Prima domanda: in quante relazioni si trovano i dati che servono?
- Serve fare la query sul numero minimo di relazioni possibile
- Il risultato: più leggibile, e più leggero per il DBMS

## Soluzione

```
SEL Stipendio>40
(Impiegati)
```

Bastano una selezione: Impiegati ha già tutti gli attributi richiesti (Matricola, Nome, Età, Stipendio).

ALGEBRA RELAZIONALE — QUERY GUIDATE

# Query 2 — Solo matricola, nome ed età

Impiegati(Matricola, Nome, Età, Stipendio)

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

Supervisione(Impiegato, Capo)

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

*"Trovare matricola, nome ed età degli impiegati che guadagnano più di 40."*

**Soluzione**

```
PROJ Matricola,Nome,Età (SEL Stipendio>40 (Impiegati))
```

La selezione resta identica a prima. Cambia solo la proiezione esterna: lo Stipendio serviva per filtrare, ma non compare più nel risultato finale.

# Query 3

*"Trovare i capi degli impiegati che guadagnano più di 40."*

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

## Soluzione

```
PROJ Capo (Supervisione JOIN Impiegato=Matricola (SEL Stipendio>40 (Impiegati)))
```

**Domanda** "Ho capi duplicati?" — Sì, un capo può comparire più volte in Supervisione JOIN Impiegati; ma PROJ elimina i duplicati, perché una relazione è un *insieme* di tuple.

ALGEBRA RELAZIONALE — QUERY GUIDATE

# L'ordine degli operatori conta

Due modi di scrivere la query 3 — sono equivalenti?

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

**1 · CORRETTA**

```
PROJ Capo (SEL Stipendio>40
(Supervisione JOIN
Impiegato=Matricola (Impiegati)))
```

**2 · ERRATA**

```
SEL Stipendio>40 (PROJ Capo
(Supervisione JOIN
Impiegato=Matricola (Impiegati)))
```

**No, non sono equivalenti.** In 2) la selezione su Stipendio agisce su PROJ Capo(...), una relazione che ha il solo attributo Capo: Stipendio non esiste più lì. È un errore sintattico.

# Query 4

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

*"Trovare nome e stipendio dei capi degli impiegati che guadagnano più di 40."*

## Soluzione

```
PROJ Nome,Stipendio (Impiegati JOIN Matricola=Capo PROJ Capo
(Supervisione JOIN Impiegato=Matricola (SEL Stipendio>40 (Impiegati))))
```

Esiste una notazione alternativa, più visiva: l'**expression tree**. La vediamo nella prossima slide, sulla stessa formula.

EXPRESSION TREE — QUERY 4

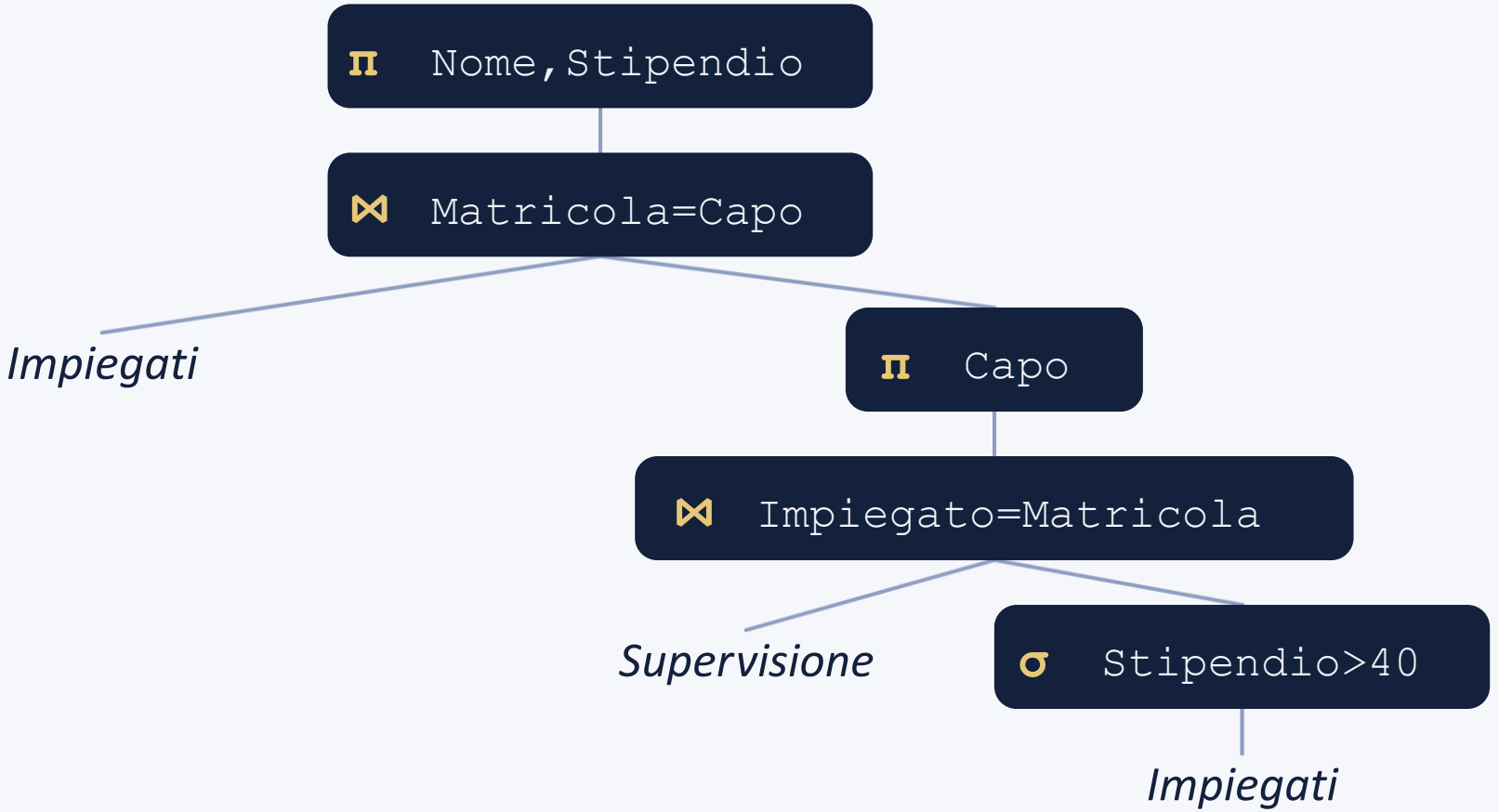
# La stessa formula, letta come albero

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |



## EXPRESSION TREE — UN CASO PIÙ DIFFICILE

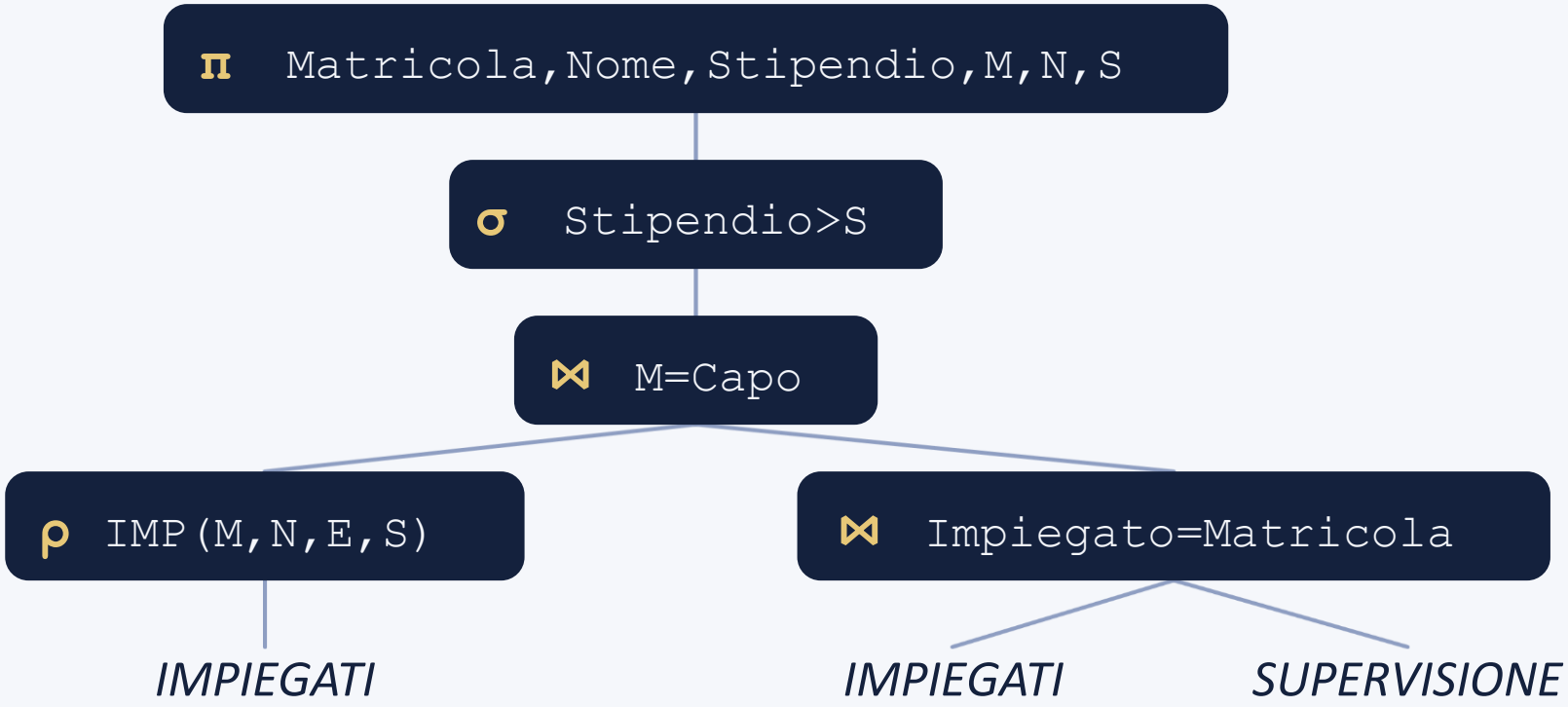
# Impiegati che guadagnano più del proprio capo

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |



# La formula completa dietro l'albero

```
PROJ Matricola,Nome,Stipendio,MatrC,NomeC,StipC (SEL Stipendio>StipC (
REN MatrC,NomeC,StipC,EtàC ← Matr,Nome,Stip,Età (Impiegati) JOIN
MatrC=Capo (Supervisione JOIN Impiegato=Matricola Impiegati)))
```

**Per il compito**

Nell'expression tree si possono usare sia le lettere greche (π) sia le label (PROJ)
All'esame si scrivono **sia l'albero sia la formula algebrica**

# Query 5

**Impiegati(Matricola, Nome, Età, Stipendio)**

| Matricola | Nome | Età | Stipendio |
|---|---|---|---|
| 7309 | Rossi | 34 | 45 |
| 5998 | Bianchi | 37 | 38 |
| 9553 | Neri | 42 | 35 |
| 5698 | Bruni | 43 | 42 |
| 4076 | Mori | 45 | 50 |
| 8123 | Lupi | 46 | 60 |

**Supervisione(Impiegato, Capo)**

| Impiegato | Capo |
|---|---|
| 7309 | 5698 |
| 5998 | 5698 |
| 9553 | 4076 |
| 5698 | 4076 |
| 4076 | 8123 |

*"Trovare le matricole dei capi i cui impiegati guadagnano **tutti** più di 40."*

**Tutti i capi**

```
PROJ Capo (Supervisione)
```

**– *(meno)***

**Capi con ≥1 impiegato ≤40**

```
PROJ Capo (Supervisione JOIN Impiegato=Matricola
(SEL Stipendio<=40 (Impiegati)))
```

**Tecnica**

L'algebra relazionale non ha un operatore "per ogni". Si simula con una doppia negazione: tutti i capi, meno quelli con almeno un'eccezione.

TOOL INTERATTIVO

# Esercizi con RelaX

### 1. Accedi al sito

https://dbis-uibk.github.io/relax

### 2. Scegli lo schema

Dal menu a tendina in alto a sinistra scegliere:

`Progetti - by Torlone`

**Schema** `Impiegato` `Dipartimento` `Progetto` `Partecipazione`

TOOL INTERATTIVO

# Importare nuovi schemi in RelaX

### 1. Apri il Group Editor

Su dbis-uibk.github.io/relax clicca **Group Editor** in alto.

### 2. Incolla il file

Apri il file .txt, copia **tutto** il contenuto e incollalo nell'editor.

### 3. Carica lo schema

Clicca **Preview**, poi **Use Group in Editor**.

**supervisione.txt** `Impiegati` `Supervisione`

**stoccaggi.txt** `Magazzino` `Azienda` `Prodotto` `Stoccaggio`

*Lo schema resta caricato solo nella sessione del browser: dopo un ricaricamento della pagina va importato di nuovo.*

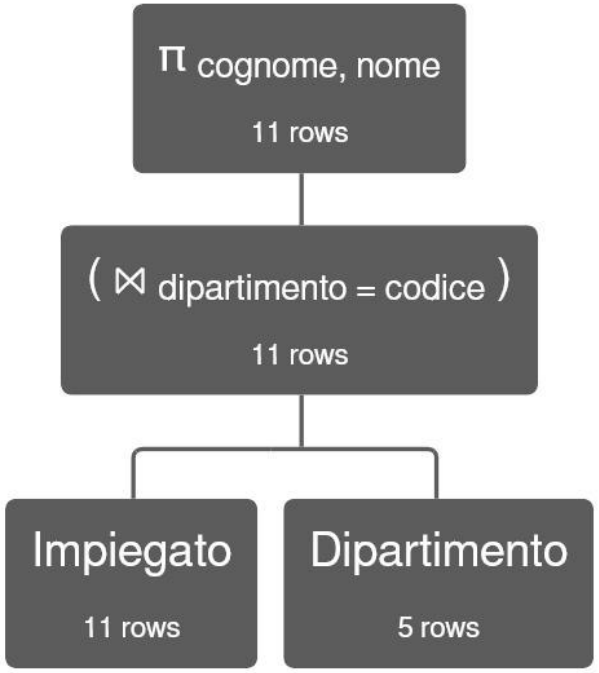
ESERCIZIO 1

## Per ogni impiegato, il suo cognome e il nome del dipartimento in cui lavora

```
π cognome,nome (
  Impiegato ⋈ dipartimento=codice Dipartimento
)
```

ALBERO DI VALUTAZIONE

# Esercizio 1 — come viene valutata la query



ESERCIZIO 2

**Per ogni progetto con bilancio superiore a 100.000 €: il nome e il bilancio del progetto, e quello degli impiegati che vi partecipano, uno per riga**

$$\pi_{\text{nome,bilancio,cognome}}(\sigma_{\text{bilancio}>100000}\,\text{Progetto} \bowtie_{\text{progetto=sigla}} \text{Partecipazione} \bowtie_{\text{impiegato=matricola}} \text{Impiegato})$$

**SOLO PROGETTI CON PARTECIPANTI: INNER JOIN**

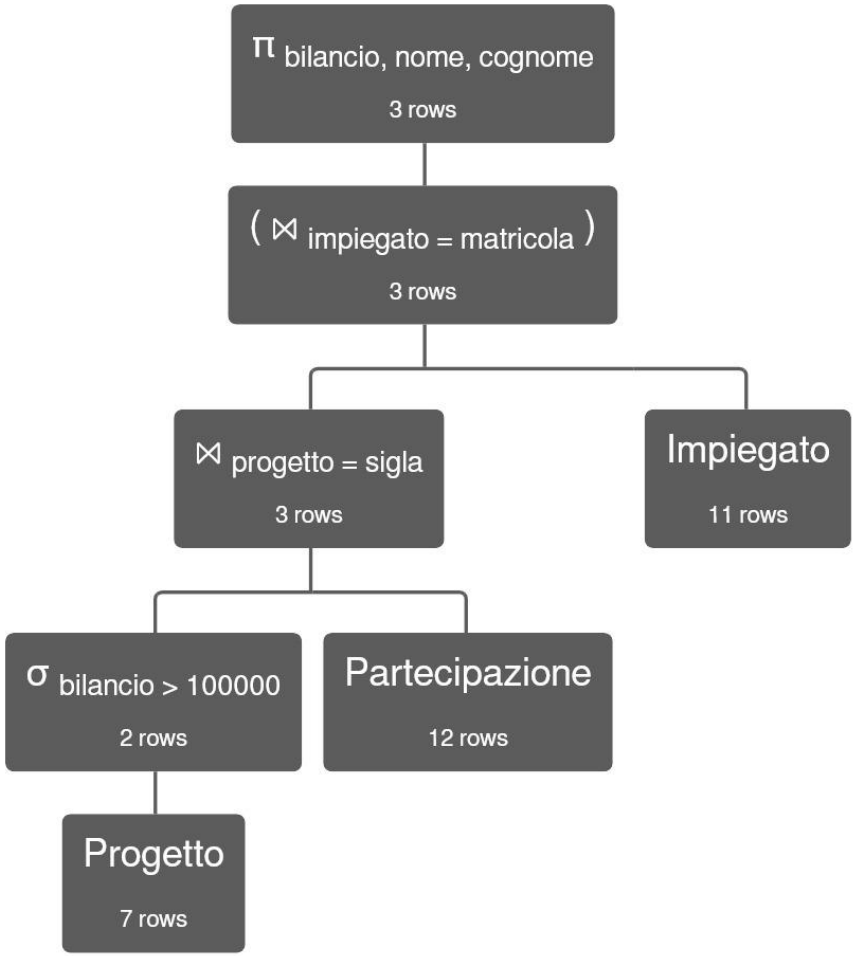
```
Progetto ⋈ sigla=progetto Partecipazione
```

**ANCHE SENZA PARTECIPANTI: JOIN SINISTRO (LEFT OUTER JOIN)**

```
Progetto ⟕ sigla=progetto Partecipazione
```

ALBERO DI VALUTAZIONE

# Esercizio 2 — come viene valutata la query



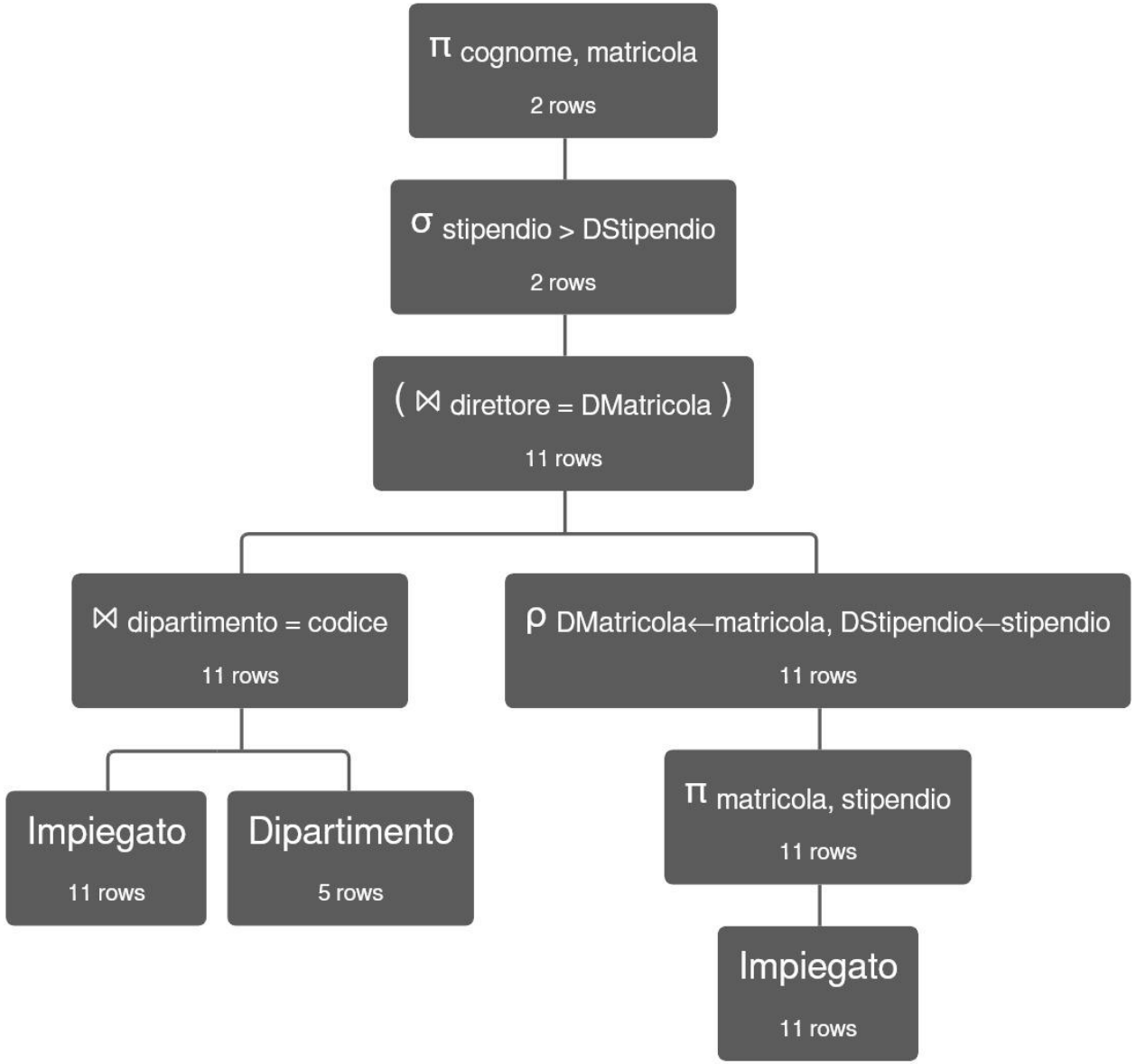
ESERCIZIO 3

**Il cognome e la matricola degli impiegati che guadagnano di più del direttore del dipartimento per cui lavorano**

$$\pi_{\text{cognome,matricola}} \Big( \sigma_{\text{stipendio}>\text{DStipendio}} \Big( \text{Impiegato} \bowtie_{\text{dipartimento=codice}} \text{Dipartimento} \bowtie_{\text{direttore=DMatricola}} \Big( \rho_{\text{DMatricola}\leftarrow\text{matricola, DStipendio}\leftarrow\text{stipendio}} \Big( \pi_{\text{matricola,stipendio}} \text{Impiegato} \Big) \Big) \Big) \Big)$$

ALBERO DI VALUTAZIONE

# Esercizio 3



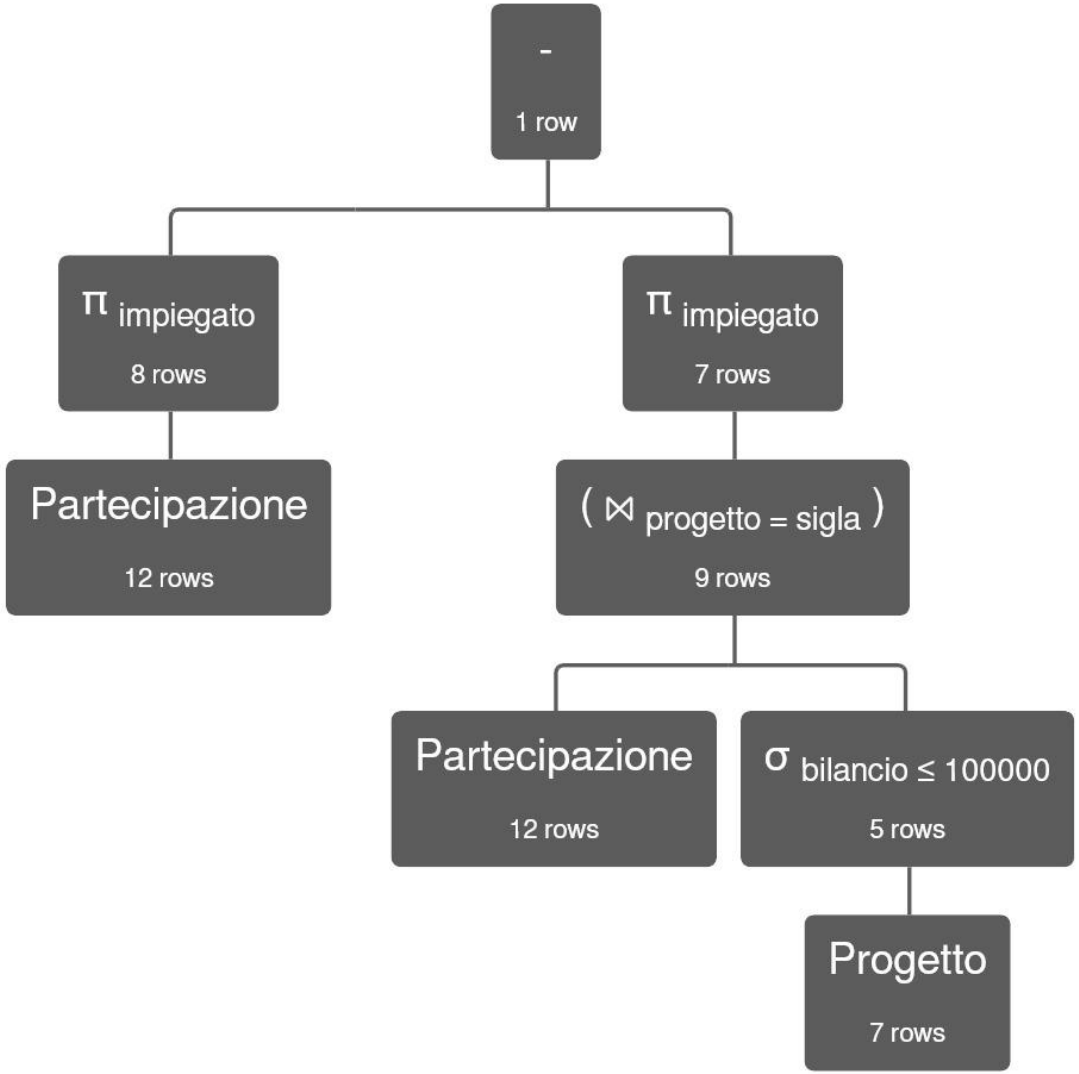
ESERCIZIO 4

**La matricola degli impiegati che partecipano solo a progetti con budget superiore a 100.000 €**

$$\pi_{\text{impiegato}}(\text{Partecipazione}) - \pi_{\text{impiegato}}(\text{Partecipazione} \bowtie_{\text{progetto}=\text{sigla}} (\sigma_{\text{bilancio} <= 100000}(\text{Progetto})))$$

ALBERO DI VALUTAZIONE

# Esercizio 4



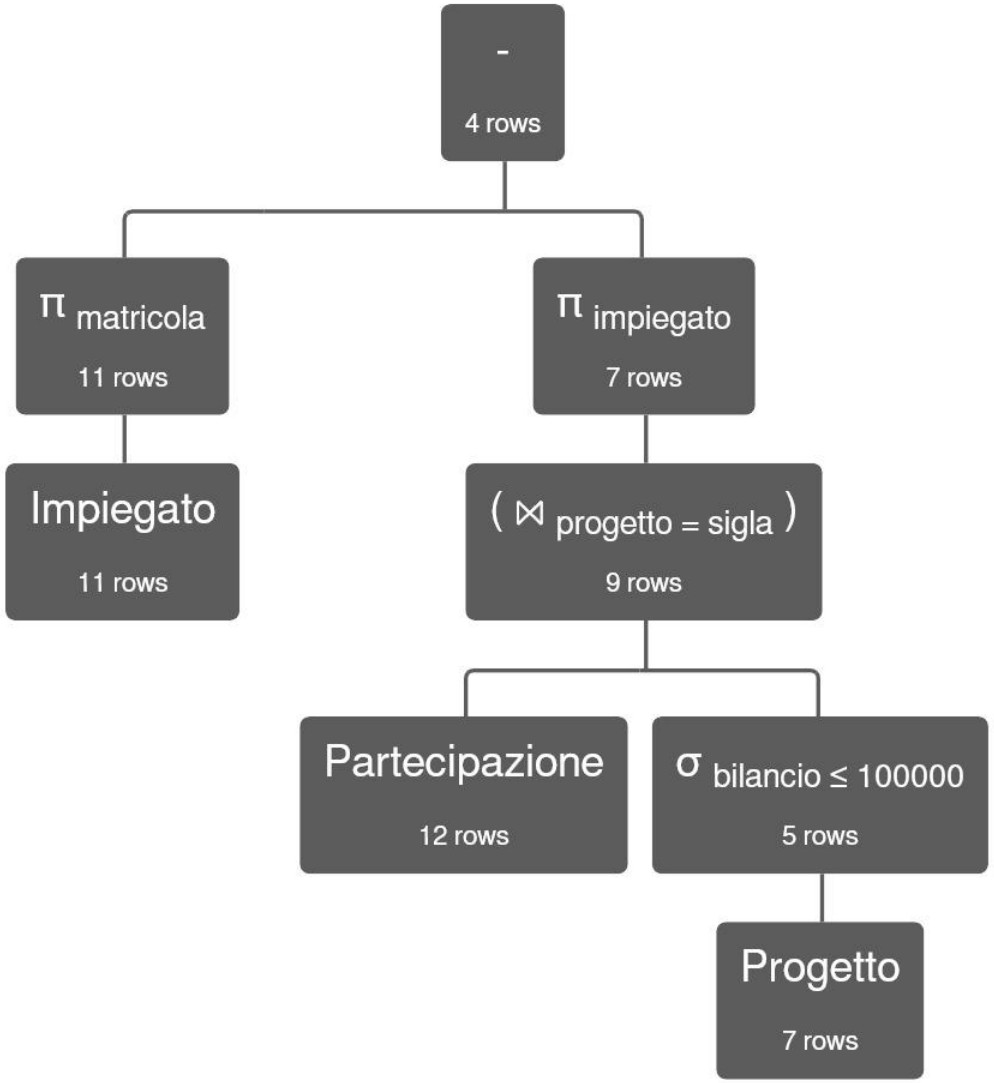
ESERCIZIO 5: ATTENTI A DETTAGLI !

**La matricola degli impiegati che non partecipano a nessun progetto con budget inferiore a 100.000 €**

$$\pi_{\text{matricola}}(\text{Impiegato}) - \pi_{\text{impiegato}}(\text{Partecipazione} \bowtie_{\text{progetto}=\text{sigla}} (\sigma_{\text{bilancio} < 100000}(\text{Progetto})))$$

ALBERO DI VALUTAZIONE

# Esercizio 5



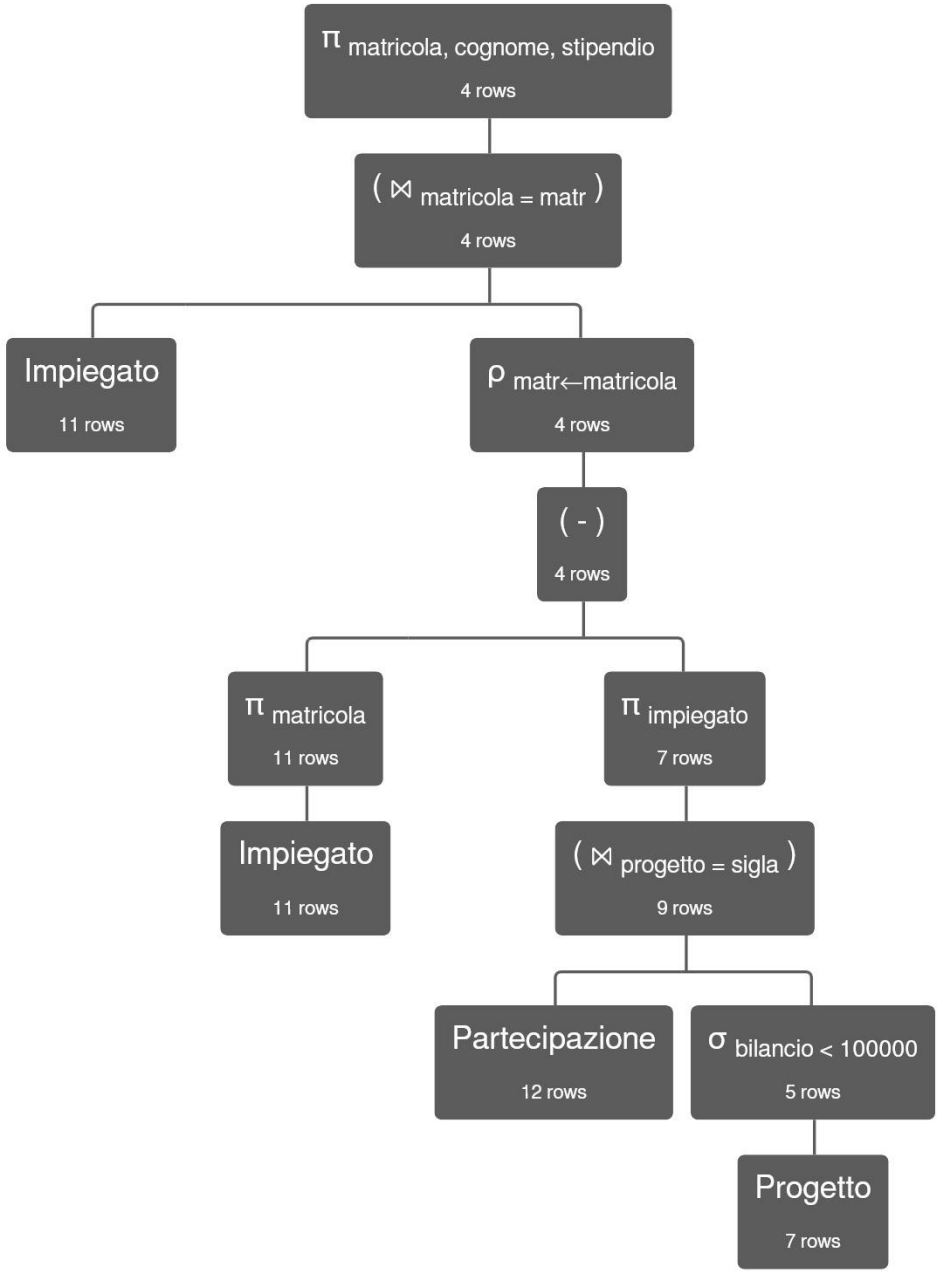
ESERCIZIO 6

**Cognome, matricula e stipendio degli impiegati che non partecipano a nessun progetto con budget inferiore a 100.000 €**

$$\pi_{\text{matricola,cognome,stipendio}}(\text{Impiegato} \bowtie_{\text{matricola=matr}} (\rho_{\text{matr} \leftarrow \text{matricola}} (\pi_{\text{matricola}} \text{Impiegato} - \pi_{\text{impiegato}} (\text{Partecipazione} \bowtie_{\text{progetto=sigla}} (\sigma_{\text{bilancio} < 100000} \text{Progetto})))))$$

ALBERO DI VALUTAZIONE

# Esercizio 6



OLTRE LE QUERY

# Viste e limiti dell'algebra relazionale

Come si definiscono relazioni derivate, l'architettura a tre livelli di un DBMS, e cosa l'algebra relazionale non riesce a esprimere.

VISTE

# Viste (relazioni derivate)

Rappresentazioni diverse per gli stessi dati: è lo **schema esterno** del DBMS.

**Relazioni di base**

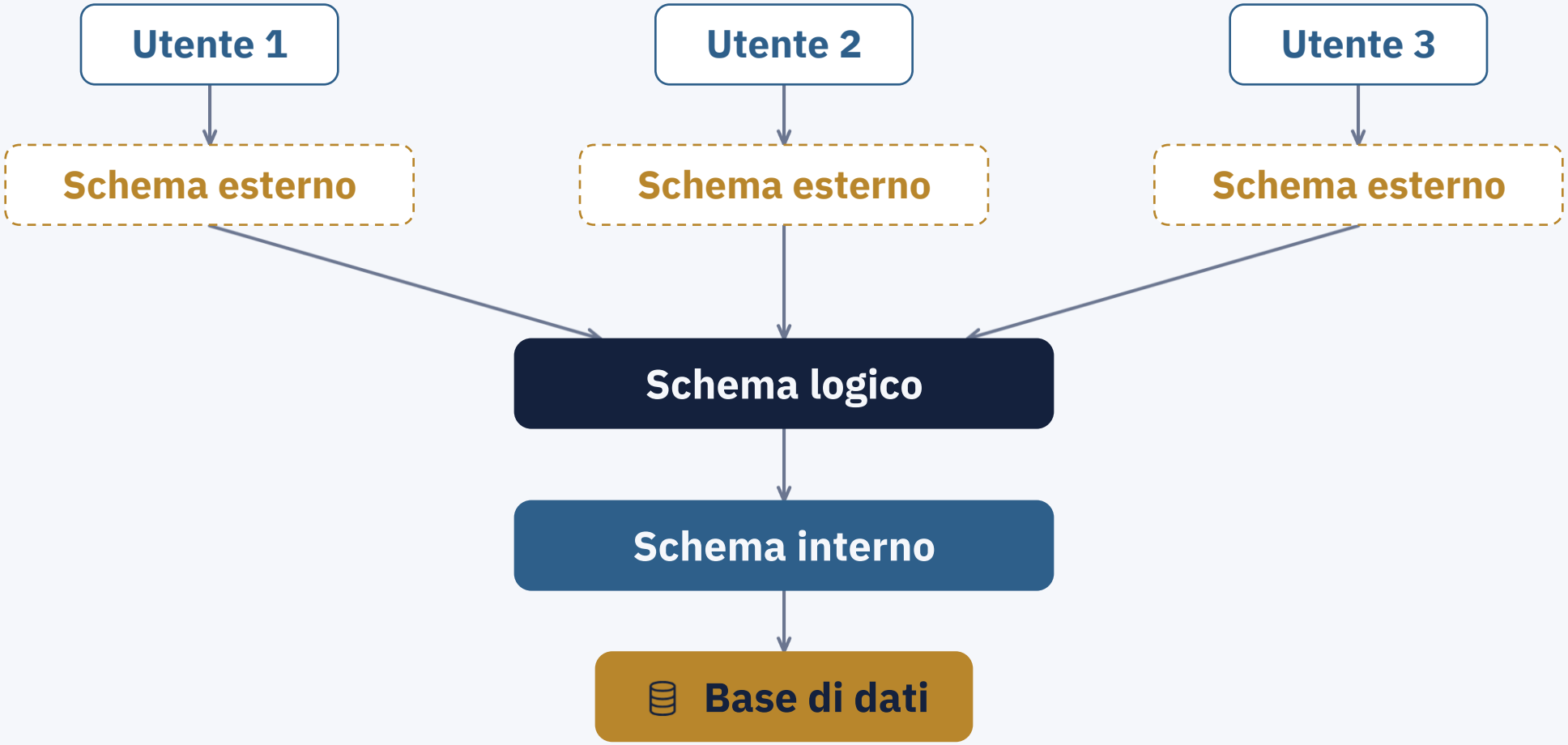
Il loro contenuto è **autonomo**: le tuple sono memorizzate direttamente nel database.

**Relazioni derivate (viste)**

Il loro contenuto è **funzione** del contenuto di altre relazioni, definito per mezzo di un'interrogazione.

Le relazioni derivate possono essere definite anche su altre relazioni derivate.

# Architettura standard a tre livelli (ANSI/SPARC)



# Viste, un esempio

**Afferenza(Impiegato, Reparto)**

| Impiegato | Reparto |
|---|---|
| Rossi | A |
| Neri | B |
| Bianchi | B |

**Direzione(Reparto, Capo)**

| Reparto | Capo |
|---|---|
| A | Mori |
| B | Bruni |

**Definizione di una vista**

```
Supervisione = PROJ Impiegato,Capo (Afferenza NAT_JOIN Direzione)
```

La relazione Supervisione usata in tutti gli esempi precedenti è, in realtà, proprio una vista come questa

# Interrogazioni sulle viste

Sono eseguite sostituendo alla vista la sua definizione.

**Query scritta sulla vista**

```
SEL Capo='Leoni' (Supervisione)
```

↓ **sostituzione**

**Query eseguita davvero**

```
SEL Capo='Leoni' (PROJ Impiegato,Capo (Afferenza JOIN Direzione))
```

VISTE

# Viste, motivazioni

### Schema esterno

Ogni utente vede solo ciò che gli interessa, e ciò che è autorizzato a vedere

### Strumento di programmazione

Si può semplificare la scrittura delle interrogazioni

### Riuso

Permettono di usare programmi esistenti anche su schemi ristrutturati

LIMITI

# Algebra relazionale: limiti

Ci sono interrogazioni interessanti che non si possono esprimere

### Calcolo di valori derivati

Possiamo solo **estrarre** valori esistenti, mai calcolarne di nuovi:

- a livello di tupla o di singolo valore — conversioni, somme, ecc.
- su insiemi di tuple — somme, medie, ecc.

### Ricorsione

Interrogazioni **inerentemente ricorsive** non sono esprimibili, come la chiusura transitiva.

LIMITI

# Chiusura transitiva

Supervisione(Impiegato, Capo) — per ogni impiegato, trovare tutti i superiori: il capo, il capo del capo, e così via.

**Supervisione**

| Impiegato | Capo |
|---|---|
| Rossi | Mori |
| Neri | Mori |
| Mori | Lupi |

**Chiusura desiderata**

| Impiegato | Superiore |
|---|---|
| Rossi | Mori |
| Rossi | Lupi |
| Neri | Mori |
| Neri | Lupi |
| Mori | Lupi |

**Non esiste, in algebra relazionale, un modo di esprimere questa interrogazione**

PER RIPASSARE

# Ripasso online

Due video su Relational Algebra, dal corso Stanford Lagunita:

- **Relational Algebra 1** — youtube.com/watch?v=tii7xcFilOA
- **Relational Algebra 2** — youtube.com/watch?v=GkBf2dZAES0

SCHEMA DI RIFERIMENTO

# Schema «Stoccaggi»

## MAGAZZINO

(CodiceM, Città, Via, NumeroCivico)

## AZIENDA

(CodiceA, NomeAzienda, Città, CAP, Telefono)

## PRODOTTO

(CodiceP, NomeProdotto, Azienda, Prezzo)

FK: Azienda → AZIENDA.CodiceA

## STOCCAGGIO

(Prodotto, Magazzino, Quantità)

FK: Prodotto → PRODOTTO.CodiceP
FK: Magazzino → MAGAZZINO.CodiceM

*Attributo sottolineato = chiave primaria · FK = chiave esterna*

**ESERCIZIO**

PRODOTTO(CodiceP, NomeProdotto, Azienda, Prezzo)

## Elencare il nome di tutti i prodotti che costano meno di 10 Euro

```
PROJ NomeProdotto (SEL Prezzo<10 (PRODOTTO))
```

**ESERCIZIO**

MAGAZZINO(CodiceM, Città, Via, NumeroCivico)

## Elencare i numeri civici dei magazzini di via Ripamonti a Milano

```
PROJ NumeroCivico (SEL Città='Milano' AND Via='Ripamonti'
(MAGAZZINO))
```

**ESERCIZIO**

STOCCAGGIO(Prodotto, Magazzino, Quantità)

## Elencare il codice dei prodotti stoccati in un magazzino avente lo stesso codice del prodotto

```
PROJ Prodotto (SEL Prodotto=Magazzino (STOCCAGGIO))
```

**ESERCIZIO**

```
PRODOTTO(CodiceP, NomeProdotto, Azienda, Prezzo)
AZIENDA(CodiceA, NomeAzienda, Città, CAP, Telefono)
```

## Elencare il nome dei prodotti delle aziende di Milano

```
PROJ NomeProdotto (PRODOTTO JOIN CodiceA=Azienda
(SEL Città='Milano' (AZIENDA)))
```

**ESERCIZIO**

STOCCAGGIO(Prodotto, Magazzino, Quantità)
PRODOTTO(CodiceP, NomeProdotto, Azienda, Prezzo)

## Elencare il codice di tutti i magazzini in cui è stoccato un prodotto di nome «iPhone» «iPhone»

```
PROJ Magazzino (STOCCAGGIO JOIN Prodotto=CodiceP
(SEL NomeProdotto='iPhone' (PRODOTTO)))
```

**ESERCIZIO**

```
MAGAZZINO(CodiceM, Città, Via, NumeroCivico)
STOCCAGGIO(Prodotto, Magazzino, Quantità)
PRODOTTO(CodiceP, NomeProdotto, Azienda, Prezzo)
```

**Elencare la città di tutti i magazzini in cui è stoccato un prodotto di nome «iPhone» «iPhone»**

```
PROJ Città (MAGAZZINO JOIN Magazzino=CodiceM
(STOCCAGGIO JOIN Prodotto=CodiceP
(SEL NomeProdotto='iPhone' (PRODOTTO))))
```

ESERCIZIO PIÙ COMPLESSO

## Elencare il codice di quei prodotti che sono stoccati nella città dell'azienda che li che li produce

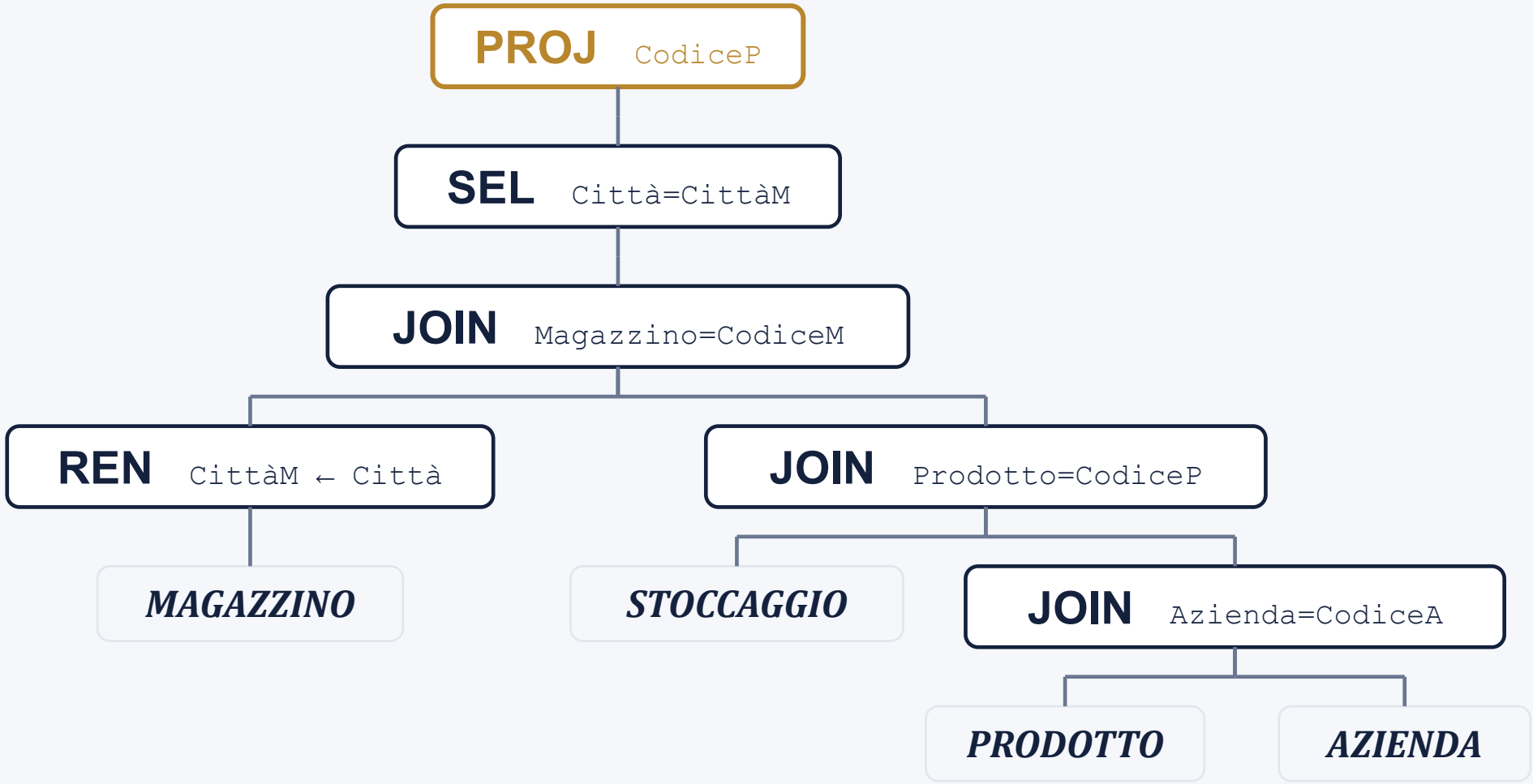


Qui la città del magazzino va confrontata con la città dell'azienda produttrice: due attributi Città diversi, lungo la catena PRODOTTO → AZIENDA e STOCCAGGIO → MAGAZZINO.

Serve una ridenominazione (REN) prima del confronto, per evitare l'ambiguità tra le due colonne Città.

ALBERO DELL'ESPRESSIONE

# Codice dei prodotti stoccati nella città dell'azienda produttrice



**SOLUZIONE**

**Elencare il codice di quei prodotti che sono stoccati nella città dell'azienda che li produce produce**

```
PROJ CodiceP (SEL Città=CittaM
(REN CittaM←Città (MAGAZZINO) JOIN Magazzino=CodiceM
(STOCCAGGIO JOIN Prodotto=CodiceP
(PRODOTTO JOIN Azienda=CodiceA (AZIENDA))))))
```

**ESERCIZIO**

PRODOTTO(CodiceP, NomeProdotto, Azienda, Prezzo)

## Elencare il codice, il nome e il prezzo dei prodotti che costano più di 100

```
PROJ CodiceP,NomeProdotto,Prezzo (SEL Prezzo>100 (PRODOTTO))
```

**ESERCIZIO**

PRODOTTO(CodiceP, NomeProdotto, Azienda, Prezzo)
AZIENDA(CodiceA, NomeAzienda, Città, CAP, Telefono)

## Elencare il nome e il prezzo dei prodotti dell'azienda di nome «Apple»

```
PROJ NomeProdotto,Prezzo (PRODOTTO JOIN Azienda=CodiceA
(SEL NomeAzienda='Apple' (AZIENDA)))
```

**ESERCIZIO PIÙ COMPLESSO**

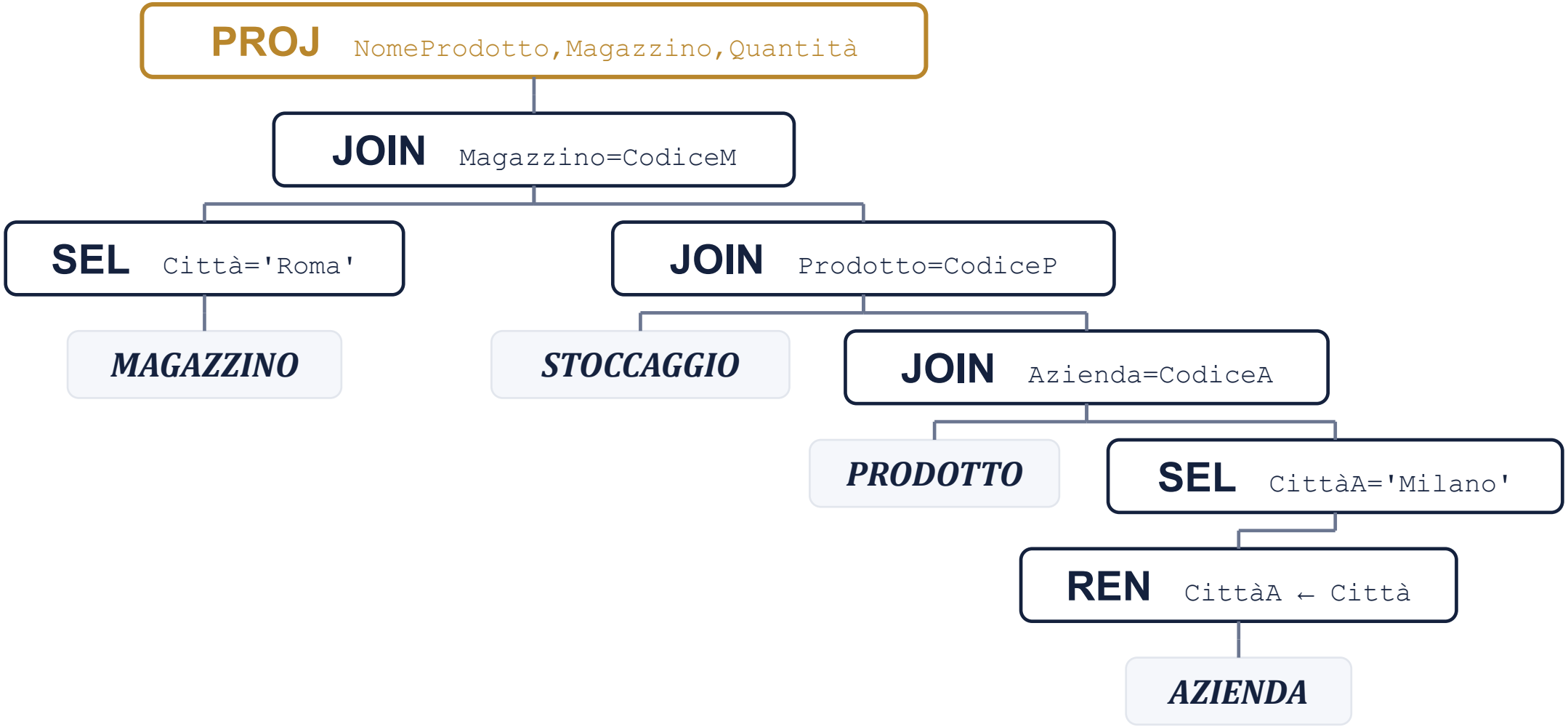
**Selezionare i prodotti di aziende milanesi stoccati in magazzini di Roma, indicando per ogni stoccaggio il nome del prodotto, il codice del magazzino e la quantità stoccata**

Tre condizioni da unire: azienda a Milano, magazzino a Roma, e il collegamento PRODOTTO → STOCCAGGIO → MAGAZZINO.

Anche qui compaiono due attributi Città: quello dell'azienda va ridenominato prima del confronto.

ALBERO DI VALUTAZIONE

# Prodotti di aziende milanesi stoccati in magazzini di Roma



**ESPRESSIONE ALGEBRICA**

## Prodotti di aziende milanesi stoccati in magazzini di Roma

```
PROJ NomeProdotto,Magazzino,Quantità ((SEL Città='Roma'
(MAGAZZINO)) JOIN Magazzino=CodiceM

STOCCAGGIO JOIN Prodotto=CodiceP

(PRODOTTO JOIN Azienda=CodiceA

(SEL CittàA='Milano' (REN CittàA←Città AZIENDA))))
```

**ESERCIZIO**

MAGAZZINO(CodiceM, Città, Via, NumeroCivico)
STOCCAGGIO(Prodotto, Magazzino, Quantità)

## Elencare il codice di tutti i magazzini in cui non è stoccato nessun prodotto

```
REN Magazzino←CodiceM (PROJ CodiceM (MAGAZZINO))
- PROJ Magazzino (SEL Quantità>0 (STOCCAGGIO))
```

**ESERCIZIO**

MAGAZZINO(CodiceM, Città, Via, NumeroCivico)
AZIENDA(CodiceA, NomeAzienda, Città, CAP, Telefono)

## Elencare le città che sono sede di un'azienda o di un magazzino (o di entrambi) entrambi)

```
PROJ Città (MAGAZZINO) U PROJ Città (AZIENDA)
```

**ESERCIZIO**

## Elencare le coppie di prodotti diversi che hanno lo stesso nome (indicando il codice di ognuno)

**VISTE AUSILIARIE**

$$\mathbf{PROD1} = \mathbf{REN}\ C1,N1,A1,P1\ (PRODOTTO)$$

$$\mathbf{PROD2} = \mathbf{REN}\ C2,N2,A2,P2\ (PRODOTTO)$$

$$\mathbf{PROJ}\ C1,C2\ (\mathbf{SEL}\ (N1=N2)\ AND\ (C1\neq C2)\ (PROD1 \times PROD2))$$

*(in questo caso l'uso di viste aiuta ma non è strettamente necessario)*

RAFFINAMENTO

## La stessa coppia esce due volte: come tenerne una sola?

```
PROD1 = REN C1,N1,A1,P1(PRODOTTO)
PROD2 = REN C2,N2,A2,P2(PRODOTTO)
```

### Con ≠

```
PROJ C1,C2 (SEL (N1=N2) AND (C1≠C2) (PROD1×PROD2))
```

⚠ Escono sia [33,66] sia [66,33]: coppie duplicate.

### Con >

```
PROJ C1,C2 (SEL (N1=N2) AND (C1>C2) (PROD1×PROD2))
```

Esce solo [66,33]: una coppia per confronto, nessun duplicato.

**ESERCIZIO**

## Elencare il codice dei prodotti stoccati in almeno 2 magazzini diversi

**VISTE AUSILIARIE**

**STOC1 = REN** P1,M1,Q1 (STOCCAGGIO)

**STOC2 = REN** P2,M2,Q2 (STOCCAGGIO)

**PROJ P1 (SEL** (P1=P2) AND (M1≠M2) (STOC1 × STOC2))

*(in questo caso l'uso di viste aiuta ma non è strettamente necessario)*

ESERCIZIO

## Elencare il prezzo del prodotto più caro

**VISTE AUSILIARIE**

```
PROD1 = REN P1←Prezzo (PROJ Prezzo (PRODOTTO))
PROD2 = REN P2←Prezzo (PROJ Prezzo (PRODOTTO))
```

```
PROJ Prezzo (PRODOTTO) −
REN Prezzo←P2 (PROJ P2 (SEL P2<P1 (PROD1×PROD2))))
```

L'algebra relazionale non ha un operatore MAX: si trova il massimo togliendo da tutti i prezzi quelli che sono minori di un altro prezzo — quel che resta è il prezzo più alto.

**ESERCIZIO**

## Elencare il nome delle aziende che producono almeno 2 prodotti diversi aventi lo stesso nome

**VISTE AUSILIARIE**

```
PROD1 = REN C1,N1,A1,P1 (PRODOTTO)

PROD2 = REN C2,N2,A2,P2 (PRODOTTO)
```

```
PROJ NomeAzienda (AZIENDA JOIN CodiceA=A1

(SEL (A1=A2) AND (N1=N2) AND (P1<P2) (PROD1×PROD2)))
```